

RECURSION

COMPUTER SCIENCE MENTORS 61A

September 30 – October 3, 2025

1 Strings Basics

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> x = 'CS61A '  
>>> y = 'CS Mentors'  
>>> x + y
```

```
>>> colors = ['red', 'yellow', 'green']  
>>> 'green' in colors
```

```
>>> 'GREEN' in colors
```

```
>>> luggage = [c + ' suitcase' for c in colors]  
>>> luggage
```

2 List Basics

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> a = [1, 2, 3]  
>>> a
```

```
>>> a[2]
```

```
>>> a[:2]
```

```
>>> b = [1, 2, 3, a, 4]
>>> b
```

```
>>> b[3][2]
```

2. Write a list comprehension that accomplishes each of the following tasks.

(a) Square all the elements of a given list, `lst`.

(b) Compute the sum of all even numbers in the list `lst`. Is there a function you can apply to a list to get the sum of all elements?

(c) Return a list of lists such that `a = [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]`.

(d) Return the same list as above, except now excluding every instance of the number 2: `b = [[0], [0, 1], [0, 1], [0, 1, 3], [0, 1, 3, 4]]`.

3 Dictionaries

1. Using a dictionary comprehension, complete the function below. It takes a single-argument function `f` and a dictionary `d`, and returns a new dictionary containing only the entries whose keys are even numbers, with `f` applied to their values.

```
def apply_to_even(f, d):
    '''
    >>> apply_to_even(lambda x: x**2, {1:3, 2:5, 4:2, 3:2})
    {2: 25, 4: 4}
    '''
    return {__ : ____ for ____ in ____ if _____}
```

4 List Recursion

1. Complete the definition for `all_true`, which takes in a list `lst` and returns `True` if there are no `False`-y values in the list and `False` otherwise. Make sure that your implementation is recursive. What is the name of the built-in Python function that does this?

```
def all_true(lst):  
    '''  
    >>> all_true([True, 1, 'True'])  
    True  
    >>> all_true([1, 0, 1])  
    False  
    >>> all_true([])  
    True  
    '''
```

2. Given a list of integers `lst`, return the maximum sum of a subset of size `n`. If `n` is greater than or equal to the length of `lst`, just return the sum of the elements `lst`.

```
def max_subset_sum(lst, n):  
    '''  
    >>> max_subset_sum([1, 2, 3, 4], 2)  
    7  
    >>> max_subset_sum([1, 4, 2, 0, 6], 3)  
    12  
    '''  
  
    if _____:  
        _____  
  
    elif _____:  
        _____  
  
    with_elem = _____ + _____  
  
    without_elem = _____  
  
    return _____
```